

Analysis Stage Report: Twitter Sentiment Analysis Project

Overview

This report details the Analysis Stage of a Twitter Sentiment Analysis project, focusing on understanding and preparing the data before model training. The primary goal of this stage is to explore the dataset, clean it, and extract meaningful insights that help in building an effective sentiment classifier. The project uses a dataset containing tweets and their associated sentiments (positive, neutral, negative) along with the selected substrings indicating the portion of text responsible for the sentiment.

Data Loading and Initial Exploration

Loads the dataset into a pandas DataFrame. Displays the first few rows for initial inspection. Key columns: textID, text, selected_text, sentiment.

Basic Data Information

Checks for total number of entries, data types, and null values. Important for identifying potential preprocessing needs (e.g., handling missing values).

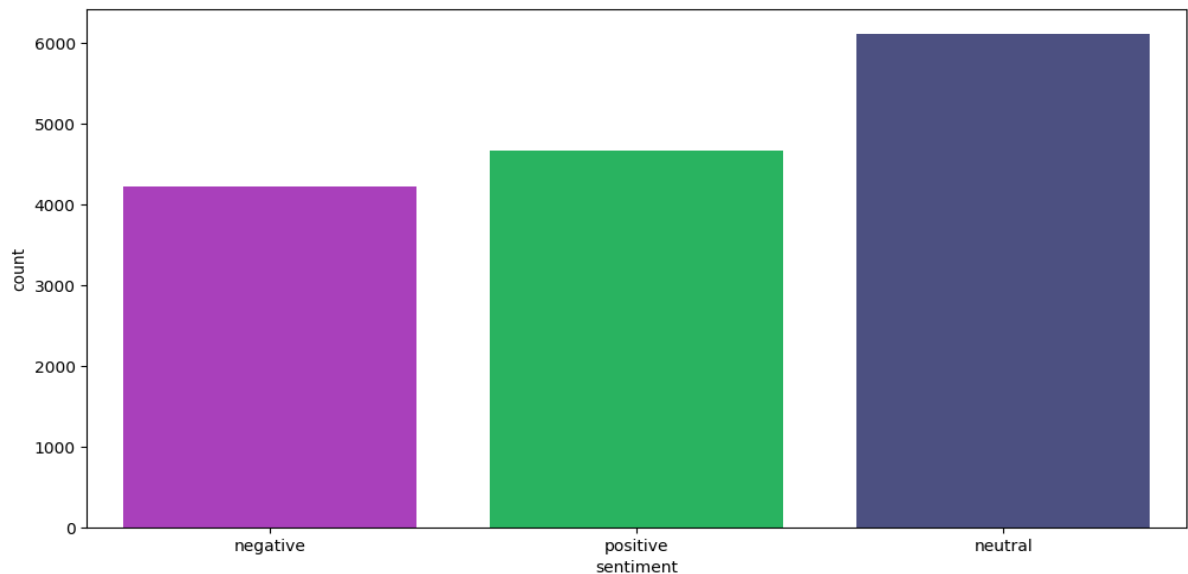
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15000 entries, 0 to 14999
Data columns (total 3 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   text            15000 non-null  object
 1   selected_text    15000 non-null  object
 2   sentiment       15000 non-null  object
dtypes: object(3)
memory usage: 351.7+ KB
```

Null Value Handling

Removes rows with null values to ensure clean data for further processing. Crucial since missing data in text or selected_text would compromise the model's learning ability.

☑ Sentiment Distribution

Visualizes class distribution using a bar chart. Shows class imbalance if present. Important for deciding strategies like stratified sampling or class-weight adjustment.



✂ Cleaning the Corpus

Make text lowercase, remove text in square brackets, remove links, remove punctuation, remove stop words and remove words containing numbers.

```
[106] def clean_text(text):
    text = str(text).lower()
    text = re.sub('\[.*?\]', '', text)
    text = re.sub('https?://\S+|www.\S+', '', text)
    text = re.sub('<.*?>+', '', text)
    text = re.sub('%s' % re.escape(string.punctuation), '', text)
    text = re.sub('\n', '', text)
    text = re.sub('\w*\d\w*', '', text)
    return text

train['text'] = train['text'].apply(lambda x:clean_text(x))
train['selected_text'] = train['selected_text'].apply(lambda x:clean_text(x))

[112] from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS

    stop_words = set(ENGLISH_STOP_WORDS)

[113] def remove_stopword(x):
    return [y for y in x if y not in stopwords.words('english')]
train['temp_list'] = train['temp_list'].apply(lambda x:remove_stopword(x))
```

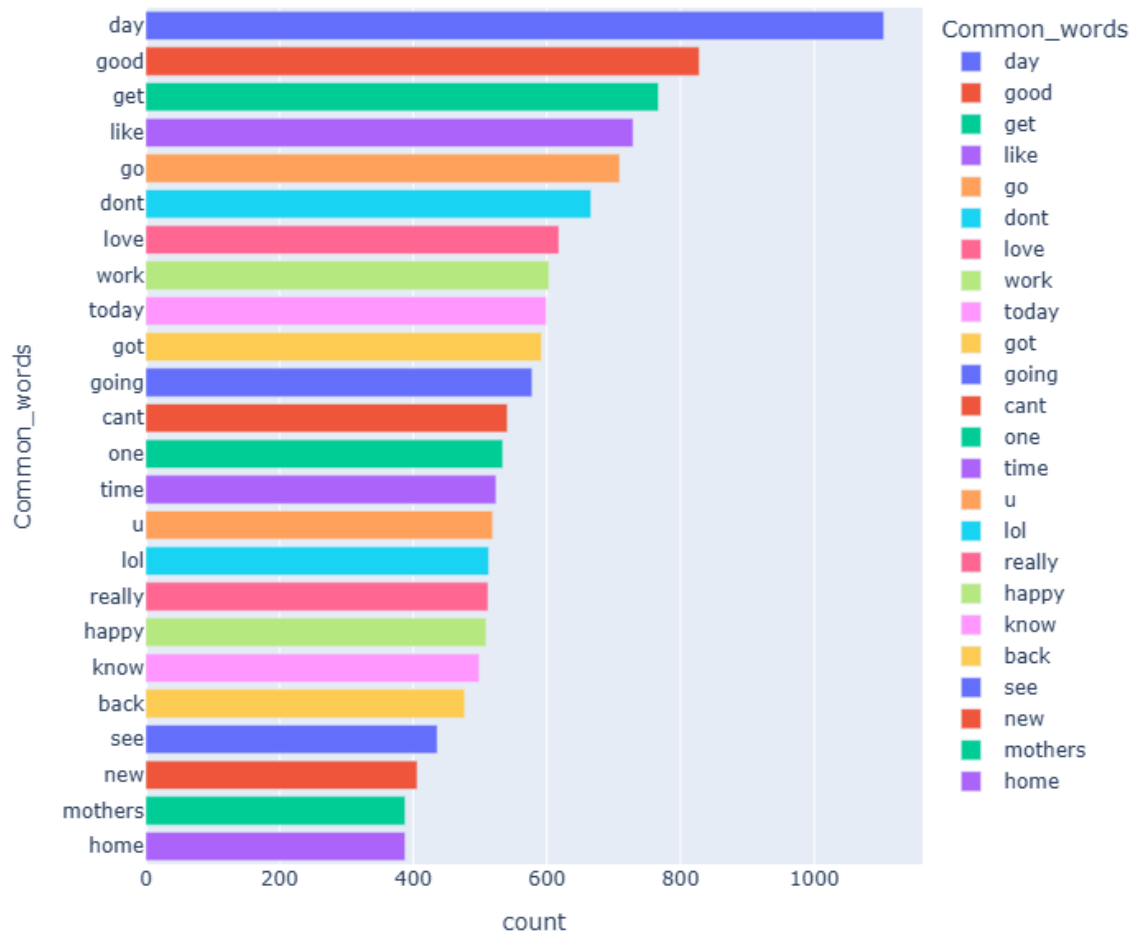
Most Common words in our Target-Selected Text

Tree of Most Common Words



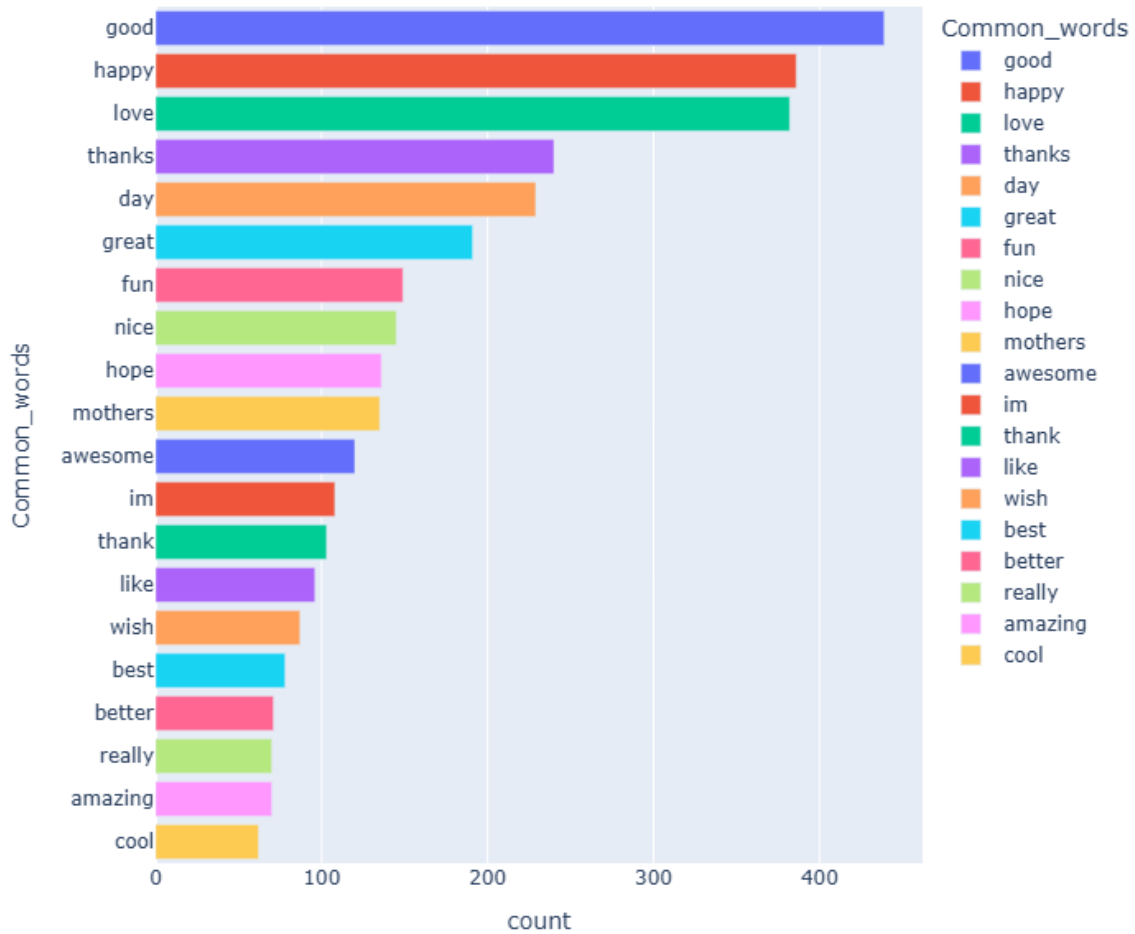
Most Common words in Text

Common Words in Text



Most common positive words

Most Common Positive Words



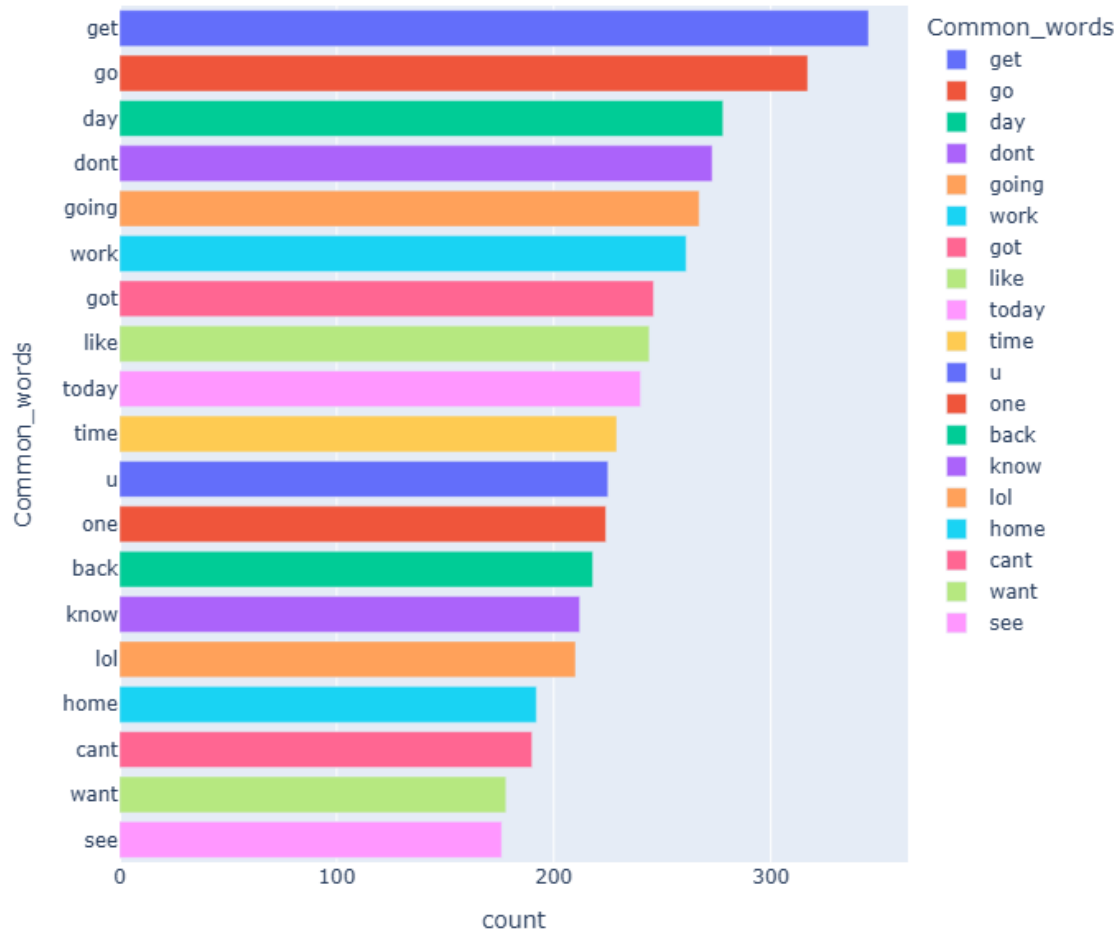
Most common Negative words

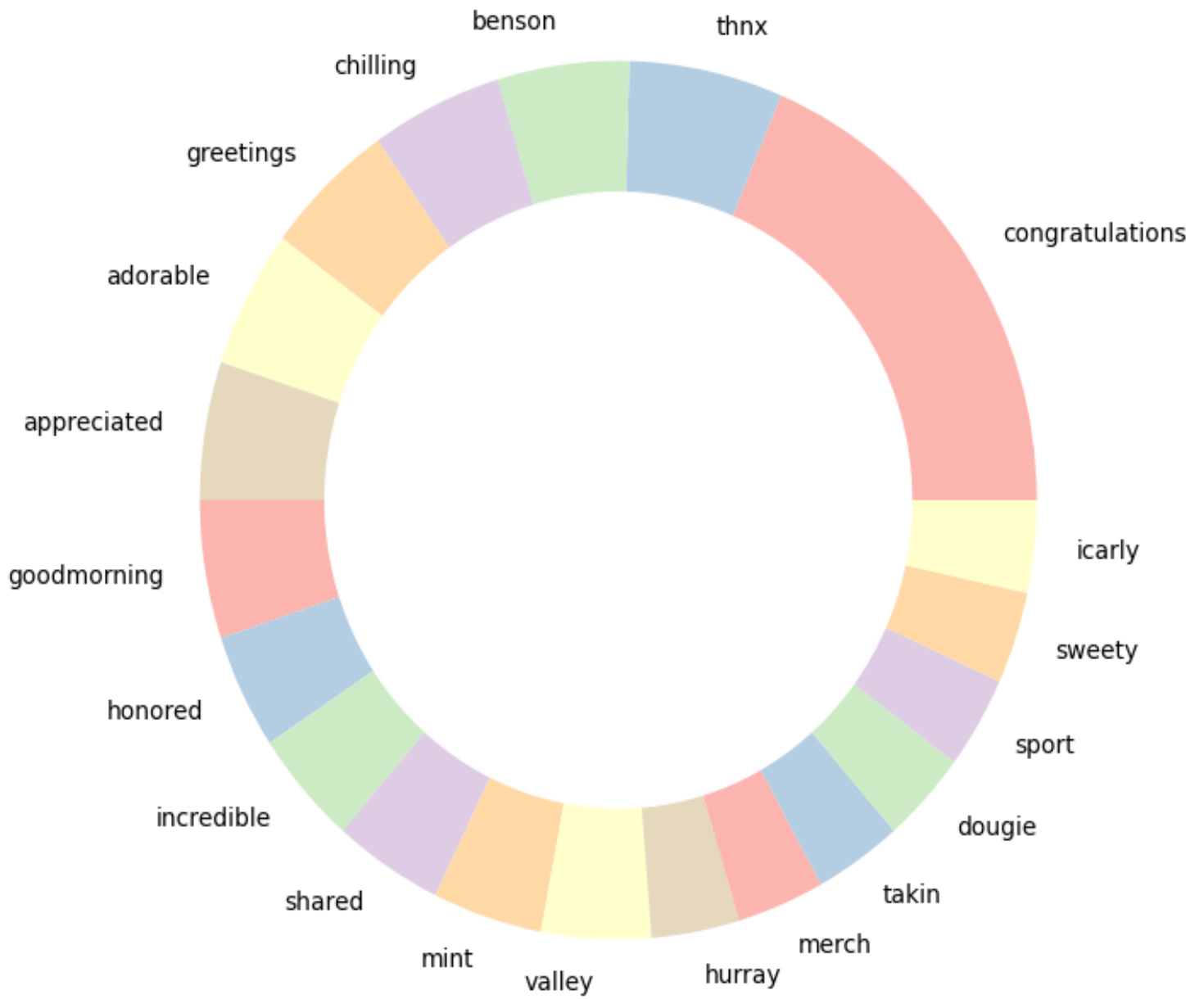
Tree Of Most Common Negative Words



Most common Neutral words

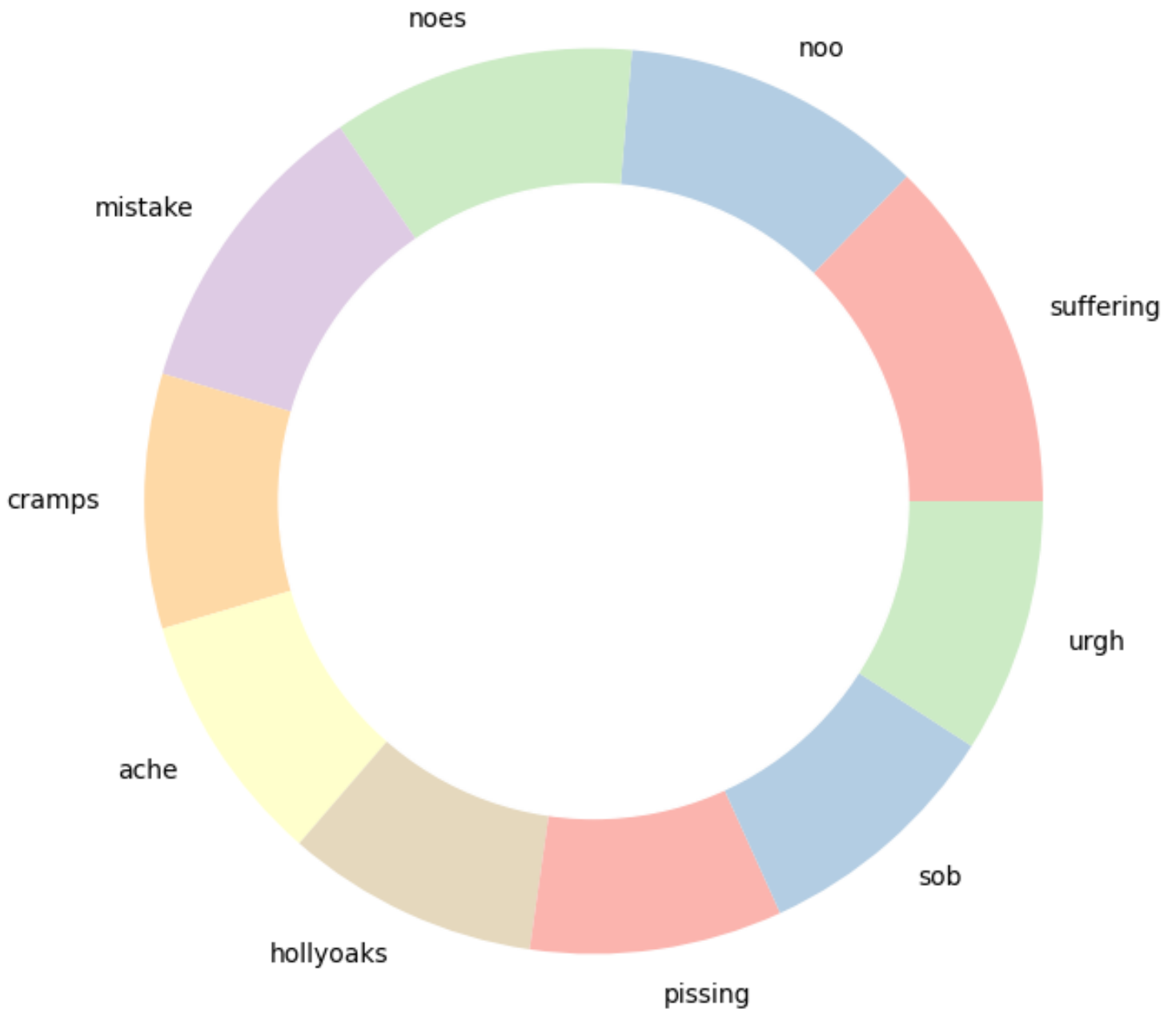
Most Common Neutral Words







Most Unique Negative Words



 Most Unique Negative Words

